

SECURITY ASSESSMENT REPORT

Project: DVWA
Date: 2026-04-18 10:30 UTC
Classification: CONFIDENTIAL — For Named Recipient Only

ASSESSMENT RESULT: REQUIRES REMEDIATION

SPI 87.9 (Hardened (CORE_GATE_FAILURE)). K-eff: 162.9. Top Risks: SAST_COMMAND_INJECTION, SAST_XSS, generic-api-key.

65.79 Security Posture Index	15.0 CRITICAL	18.0 HIGH	5.0 MEDIUM	7.0 LOW	0.0 INFO
---------------------------------	------------------	--------------	---------------	------------	-------------

Scope & Methodology

This report presents the findings of an automated security assessment conducted using Auditor Core Enterprise. The assessment combined Static Application Security Testing (SAST), Software Composition Analysis (SCA), Infrastructure-as-Code (IaC) scanning, secret detection, and CI/CD pipeline analysis. AI-assisted validation was applied to reduce false positives. Findings are mapped to SOC 2 Trust Services Criteria, CIS Controls v8, and ISO/IEC 27001:2022 Annex A.

Security Posture Index (SPI): Calculated using Weighted Security Posture Model v2.2 (WSPM). Only production and infrastructure findings are included — test files and documentation are excluded from the score. Each rule category is exposure-capped to prevent score distortion from a single noisy detector. A high SPI alongside critical findings indicates that critical issues are isolated in scope — they require immediate remediation regardless of the overall index.

Intended Use: This report may be used as supporting evidence for SOC 2 readiness preparation, ISO 27001 gap analysis, and cyber insurance underwriting pre-assessment. It does not constitute a formal audit opinion and should be reviewed by a qualified security professional.

Risk Dashboard

65.79	C	Exposure: 65.96 K-Factor: 157.55 Core/Prod: 93.3% Test/Noise: 6.7% AI Excluded: 3.0 findings
Security Posture Index	Grade	Calculation Parameters

Severity Distribution

Severity	Count	Indicator
CRITICAL	15	
HIGH	18	
MEDIUM	5	
LOW	7	
INFO	0	—

Compliance Framework Coverage

Framework	Controls Triggered	Top Affected Control	Findings
SOC 2 TSC	5	CC7.1 — Vulnerability Detection	37
CIS Controls v8	6	CIS-16.1 — Establish Secure Application Development Proc	39
ISO 27001:2022	7	A.8.26 — Application Security Requirements	33

Key Findings

The following findings represent the highest-risk items identified during the assessment, sorted by severity. Each finding includes its compliance framework mapping for remediation prioritisation.

#1 CRITICAL	generic-api-key	vulnerabilities/csrf/help/help.php:54
Description: SECRET EXPOSURE: Generic API Key identified. AI Verdict: SUPPORTED — The provided code snippet appears to be a part of a web application, specifically a change password form. The 'user-token' header contains a hardcoded token, which could be used as an API key. In the SOC2: CC6.1 · CC6.7 · C1.1 CIS: CIS-3.11 · CIS-4.7 · CIS-16.1 ISO: A.5.17 · A.8.12 · A.8.24		
#2 CRITICAL	SAST_COMMAND_INJECTION	vulnerabilities/view_help.php — 2 instances · lines 20, 22
Description: Potential command injection detected in php code. AI Verdict: SUPPORTED — The code is vulnerable to command injection due to the use of the `eval` function with user-controlled input from the `\$_GET['locale']` variable. This allows an attacker to inject malicious PHP code. SOC2: CC6.6 · CC7.1 CIS: CIS-16.1 · CIS-16.12 · CIS-7.5 ISO: A.8.28 · A.8.26 · A.8.29		
#3 CRITICAL	SAST_COMMAND_INJECTION	lnerabilities/exec/source/impossible.php — 2 instances · lines 22, 26
Description: Potential command injection detected in php code. AI Verdict: SUPPORTED — The code is vulnerable to command injection because the \$target variable is directly concatenated into the shell_exec command without any sanitization or validation. This allows an attacker to inject SOC2: CC6.6 · CC7.1 CIS: CIS-16.1 · CIS-16.12 · CIS-7.5 ISO: A.8.28 · A.8.26 · A.8.29		
#4 CRITICAL	SAST_COMMAND_INJECTION	vulnerabilities/exec/source/high.php — 2 instances · lines 26, 30
Description: Potential command injection detected in php code. AI Verdict: SUPPORTED — The code is vulnerable to command injection because it directly executes the 'ping' command with user-controlled input (\$target) without proper sanitization or validation. This allows an attacker to i SOC2: CC6.6 · CC7.1 CIS: CIS-16.1 · CIS-16.12 · CIS-7.5 ISO: A.8.28 · A.8.26 · A.8.29		
#5 CRITICAL	SAST_COMMAND_INJECTION	vulnerabilities/exec/source/medium.php — 2 instances · lines 19, 23
Description: Potential command injection detected in php code. AI Verdict: SUPPORTED — The code is vulnerable to command injection because it directly executes the 'ping' command with user-controlled input (\$target) without proper sanitization or validation. An attacker could inject mal SOC2: CC6.6 · CC7.1 CIS: CIS-16.1 · CIS-16.12 · CIS-7.5 ISO: A.8.28 · A.8.26 · A.8.29		

#6 CRITICAL	SAST_COMMAND_INJECTION	vulnerabilities/exec/source/low.php — 2 instances · lines 10, 14
Description: Potential command injection detected in php code. AI Verdict: SUPPORTED — The code is vulnerable to command injection because it directly executes the 'ping' command with user-controlled input (\$target) using shell_exec(). This allows an attacker to inject malicious command SOC2: CC6.6 · CC7.1 CIS: CIS-16.1 · CIS-16.12 · CIS-7.5 ISO: A.8.28 · A.8.26 · A.8.29		
#7 CRITICAL	SAST_COMMAND_INJECTION	vulnerabilities/javascript/source/high.js:1
Description: Potential command injection detected in javascript code. AI Verdict: NOT_SUPPORTED — The provided code is heavily obfuscated and does not clearly demonstrate a command injection vulnerability. The code appears to be implementing a SHA-256 hash function, and there is no clear indicatio SOC2: CC6.6 · CC7.1 CIS: CIS-16.1 · CIS-16.12 · CIS-7.5 ISO: A.8.28 · A.8.26 · A.8.29		
#8 CRITICAL	SAST_COMMAND_INJECTION	s/javascript/source/high_unobfuscated.js — 2 instances · lines 83, 84
Description: Potential command injection detected in javascript code. AI Verdict: NOT_SUPPORTED — The provided code does not demonstrate a command injection vulnerability. The code is using the `crypto` module to create a hash, and the `eval` function is used to require the `crypto` and `buffer` m SOC2: CC6.6 · CC7.1 CIS: CIS-16.1 · CIS-16.12 · CIS-7.5 ISO: A.8.28 · A.8.26 · A.8.29		
#9 CRITICAL	SAST_COMMAND_INJECTION	ulnerabilities/cryptography/source/oracle_attack.php:57
Description: Potential command injection detected in php code. AI Verdict: SUPPORTED — The provided code is part of a vulnerability chain (CHAIN_0003) with a critical risk. The 'curl_exec' function is used to execute a curl request, and the '\$url' variable is not sanitized. If an attack SOC2: CC6.6 · CC7.1 CIS: CIS-16.1 · CIS-16.12 · CIS-7.5 ISO: A.8.28 · A.8.26 · A.8.29		
#10 HIGH	SAST_XSS	vulnerabilities/api/source/medium.php — 2 instances · lines 14, 25
Description: Potential xss detected in php code. SOC2: CC6.6 · CC7.1 CIS: CIS-16.12 · CIS-16.1 ISO: A.8.28 · A.8.26		
#11 HIGH	SAST_XSS	vulnerabilities/api/source/low.php — 4 instances · lines 26, 34, 81, 94
Description: Potential xss detected in php code. SOC2: CC6.6 · CC7.1 CIS: CIS-16.12 · CIS-16.1 ISO: A.8.28 · A.8.26		
#12 HIGH	SAST_XSS	vulnerabilities/authbypass/authbypass.js — 4 instances · lines 43, 45, 47, 49
Description: Potential xss detected in javascript code. SOC2: CC6.6 · CC7.1 CIS: CIS-16.12 · CIS-16.1 ISO: A.8.28 · A.8.26		

#13 HIGH	SAST_XSS	vulnerabilities/xss_d/index.php — 6 instances · lines 53, 54, 57, 58, 59, 60
Description: Potential xss detected in php code. SOC2: CC6.6 · CC7.1 CIS: CIS-16.12 · CIS-16.1 ISO: A.8.28 · A.8.26		
#14 HIGH	SAST_XSS	vulnerabilities/csp/source/high.js:9
Description: Potential xss detected in javascript code. SOC2: CC6.6 · CC7.1 CIS: CIS-16.12 · CIS-16.1 ISO: A.8.28 · A.8.26		
#15 HIGH	SAST_XSS	vulnerabilities/csp/source/impossible.js:9
Description: Potential xss detected in javascript code. SOC2: CC6.6 · CC7.1 CIS: CIS-16.12 · CIS-16.1 ISO: A.8.28 · A.8.26		
#16 MEDIUM	GHA_UNPINNED_ACTION	.github/workflows/vulnerable.yml:1
Description: Unpinned action 'actions/checkout@v6.0.2'. Use a full SHA hash to prevent supply chain attacks. SOC2: CC7.1 CIS: CIS-16.12 ISO: A.8.29		
#17 MEDIUM	GHA_UNPINNED_ACTION	.github/workflows/shiftleft-analysis.yml:1
Description: Unpinned action 'actions/checkout@v6.0.2'. Use a full SHA hash to prevent supply chain attacks. SOC2: CC7.1 CIS: CIS-16.12 ISO: A.8.29		
#18 MEDIUM	GHA_UNPINNED_ACTION	.github/workflows/codeql-analysis.yml:1
Description: Unpinned action 'actions/checkout@v6.0.2'. Use a full SHA hash to prevent supply chain attacks. SOC2: CC7.1 CIS: CIS-16.12 ISO: A.8.29		
#19 MEDIUM	GHA_UNPINNED_ACTION	.github/workflows/pytest.yml:1
Description: Unpinned action 'actions/checkout@v6.0.2'. Use a full SHA hash to prevent supply chain attacks. SOC2: CC7.1 CIS: CIS-16.12 ISO: A.8.29		
#20 MEDIUM	GHA_UNPINNED_ACTION	.github/workflows/docker-image.yml:1
Description: Unpinned action 'actions/checkout@v6.0.2'. Use a full SHA hash to prevent supply chain attacks. SOC2: CC7.1 CIS: CIS-16.12 ISO: A.8.29		

Attack Path Analysis (Critical Chains)

Chain ID: CHAIN_0003

Logic: secret_to_command_injection

generic-api-key vulnerabilities/csrf/help/help.php:54	→	SAST_COMMAND_INJECTION vulnerabilities/view_help.php:20	→	SAST_COMMAND_INJECTION vulnerabilities/view_help.php:22	→	SAST_COMMAND_INJECTION vulnerabilities/exec/source/high.php:26	→	SAST_COMMAND_INJECTION vulnerabilities/cryptography/source/oracle_attack.php:57
---	---	---	---	---	---	--	---	---

Remediation Roadmap

Priority	Findings	Recommended Action	Timeline
CRITICAL	15	Immediate — patch or disable affected component within 24–72 hours. Notify stakeholders.	0–3 days
HIGH	18	Short-term — remediate within current sprint. Apply input validation, dependency patches, or configuration hardening.	1–2 weeks
MEDIUM	5	Planned — schedule in next release cycle. Review code patterns and update secure coding guidelines.	1 month
LOW	7	Backlog — address during regular maintenance. Consider adding automated checks to CI pipeline.	Next quarter

SOC 2 Control Exposure Detail

Criteria	Title	Findings Mapped
CC7.1	Vulnerability Detection	37
CC6.6	External Threat Mitigation	33
CC6.1	Logical and Physical Access Controls	8
CC6.7	Data Transmission Protection	7
C1.1	Confidentiality Identification	7

Assessment Attestation

This security assessment of **DVWA** was conducted using Auditor Core Enterprise on 2026-04-18 10:30 UTC. The assessment was performed using automated static analysis, dependency scanning, secret detection, and AI-assisted validation. Results reflect the state of the codebase at the time of scanning and should be treated as point-in-time findings.

This document is intended for use by the named organisation's security and compliance teams, and may be shared with authorised third parties including external auditors and cyber insurance underwriters under appropriate non-disclosure agreements.

Auditor Core Enterprise | Automated Security Assessment Platform

Report integrity is verified via SHA-256 hash of finding content.

Assessment conducted in accordance with WSPM v2.2 methodology. This report does not constitute a formal SOC 2 audit opinion. For SOC 2 Type I/II certification, engage a licensed CPA firm.

Prepared by / Assessment Lead

Reviewed by / Security Officer

Date: _____

Date: _____

Evidence Appendix

This appendix provides source-level evidence for 33 finding(s): all AI-confirmed (SUPPORTED) findings and any CRITICAL/HIGH findings not covered by AI analysis. Code is presented with ±5/10 line context; the flagged line is highlighted.

#1 CRITICAL	generic-api-key	vulnerabilities/csrf/help/help.php:54
Status: AI CONFIRMED SOC2: CC6.1 · CC6.7 · C1.1 CIS: CIS-3.11 · CIS-4.7 · CIS-16.1 ISO: A.5.17 · A.8.12 · A.8.24 AI Analysis: The provided code snippet appears to be a part of a web application, specifically a change password form. The 'user-token' header contains a hardcoded token, which could be used as an API key. In the context of the 'secret_to_command_injection' chain, this token could potentially be used to exploit a command injection vulnerability. The chain conte... Chain ID: CHAIN_0003 Chain Risk: CRITICAL Rule: secret_to_command_injection		
49 <pre><code>POST /vulnerabilities/csrf/ HTTP/1.1 50 Host: dvwa.test 51 Content-Length: 51 52 Content-Type: application/json 53 Cookie: PHPSESSID=0hr9ikmo07thlcvjv3u3pkfeni; security=high 54 user-token: 026d0caed93471b507ed460ebddb096 55 56 {"password_new":"a","password_conf":"a","Change":1}</pre></code> 57 58
 59 60 <h3>Impossible Level</h3> 61 <p>At this level, the site requires the user to give their current password as well as the new password. As the 62 attacker does not know this, the site is 63 </div></td> 64 </tr> 65 </table>		

#2 CRITICAL

SAST_COMMAND_INJECTION

vulnerabilities/view_help.php:20

Status: AI CONFIRMED

SOC2: CC6.6 · CC7.1 CIS: CIS-16.1 · CIS-16.12 · CIS-7.5 ISO: A.8.28 · A.8.26 · A.8.29

AI Analysis: The code is vulnerable to command injection due to the use of the `eval` function with user-controlled input from the `\$\_GET['locale']` variable. This allows an attacker to inject malicious PHP code. The chain context indicates that this finding is part of a vulnerability chain, which increases the exploitability of the issue.

Chain ID: CHAIN_0003 | Chain Risk: CRITICAL | Rule: secret_to_command_injection

```
15  $security = $_GET[ 'security' ];
16  $locale = $_GET[ 'locale' ];
17
18  ob_start();
19  if ($locale == 'en') {
20    eval( '?>' . file_get_contents( DVWA_WEB_PAGE_TO_ROOT . "vulnerabilities/{\$id}/help/help.php" ) . '<?php ' );
21  } else {
22    eval( '?>' . file_get_contents( DVWA_WEB_PAGE_TO_ROOT . "vulnerabilities/{\$id}/help/help.{\$locale}.php" ) .
      '<?php ' );
23  }
24  $help = ob_get_contents();
25  ob_end_clean();
26  } else {
27    $help = "<p>Not Found</p>";
28  }
29
30  $page[ 'body' ] .= "
```

#3 CRITICAL

SAST_COMMAND_INJECTION

vulnerabilities/view_help.php:22

Status: AI CONFIRMED

SOC2: CC6.6 · CC7.1 CIS: CIS-16.1 · CIS-16.12 · CIS-7.5 ISO: A.8.28 · A.8.26 · A.8.29

AI Analysis: This finding is similar to the previous one, with the same vulnerability and chain context. The use of `eval` with user-controlled input from the `\$\_GET['locale']` variable makes the code vulnerable to command injection. The chain context indicates that this finding is part of a vulnerability chain, which increases the exploitability of the issue.

Chain ID: CHAIN_0003 | Chain Risk: CRITICAL | Rule: secret_to_command_injection

```
17
18  ob_start();
19  if ($locale == 'en') {
20    eval( '??' . file_get_contents( DVWA_WEB_PAGE_TO_ROOT . "vulnerabilities/{\$id}/help/help.php" ) . '<?php ' );
21  } else {
22    eval( '??' . file_get_contents( DVWA_WEB_PAGE_TO_ROOT . "vulnerabilities/{\$id}/help/help.{\$locale}.php" ) .
23      '<?php ' );
24  }
25  $help = ob_get_contents();
26  ob_end_clean();
27  } else {
28    $help = "<p>Not Found</p>";
29  }
30
31  $page[ 'body' ] .= "
32  <script src='/vulnerabilities/help.js'></script>
33  <link rel='stylesheet' type='text/css' href='/vulnerabilities/help.css' />
```

#4 HIGH

SAST_XSS

vulnerabilities/api/source/medium.php:14

Status: REQUIRES REVIEW (no AI coverage)

SOC2: CC6.6 · CC7.1 CIS: CIS-16.12 · CIS-16.1 ISO: A.8.28 · A.8.26

```
9  console.log(user_json);
10  var user_info = document.getElementById ( 'user_info' );
11  var name_input = document.getElementById ( 'name' );
12
13  if (user_json.name == '') {
14    user_info.innerHTML = 'User details: unknown user';
15    name_input.value = 'unknown';
16  } else {
17    var level = 'unknown';
18    if (user_json.level == 0) {
19      level = 'admin';
20    }
21    successDiv = document.getElementById ( 'message' );
22    successDiv.style.display = 'block';
23  } else {
24    level = 'user';
25  }
```

#5HIGH

SAST_XSS

vulnerabilities/api/source/medium.php:25

Status: REQUIRES REVIEW (no AI coverage)

SOC2: CC6.6 · CC7.1 CIS: CIS-16.12 · CIS-16.1 ISO: A.8.28 · A.8.26

```
20  successDiv = document.getElementById ('message');
21  successDiv.style.display = 'block';
22  } else {
23    level = 'user';
24  }
25  user_info.innerHTML = 'User details: ' + user_json.name + ' (' + level + ')';
26  name_input.value = user_json.name;
27  }
28  }
29
30  function get_user() {
31    const url = '' . $stripped_url . "/vulnerabilities/api/v2/user/2";
32
33    fetch(url, {
34      method: 'GET',
35    })
```

#6HIGH

SAST_XSS

vulnerabilities/api/source/low.php:26

Status: REQUIRES REVIEW (no AI coverage)

SOC2: CC6.6 · CC7.1 CIS: CIS-16.12 · CIS-16.1 ISO: A.8.28 · A.8.26

```
21  console.log(user_json);
22  var user_info = document.getElementById ('user_info');
23  var name_input = document.getElementById ('name');
24
25  if (user_json.name == '') {
26    user_info.innerHTML = 'User details: unknown user';
27    name_input.value = 'unknown';
28  } else {
29    if (user_json.level == 0) {
30      level = 'admin';
31    } else {
32      level = 'user';
33    }
34    user_info.innerHTML = 'User details: ' + user_json.name + ' (' + level + ')';
35    name_input.value = user_json.name;
36  }
```

#7 HIGHSAST_XSSvulnerabilities/api/source/low.php:34

Status: REQUIRES REVIEW (no AI coverage)

SOC2: CC6.6 · CC7.1 CIS: CIS-16.12 · CIS-16.1 ISO: A.8.28 · A.8.26

```
29   if (user_json.level == 0) {
30     level = 'admin';
31   } else {
32     level = 'user';
33   }
34   user_info.innerHTML = 'User details: ' + user_json.name + ' (' + level + ')';
35   name_input.value = user_json.name;
36 }
37
38 const message_line = document.getElementById ('message');
39 if (user_json.id == 2 && user_json.level == 0) {
40   message_line.style.display = 'block';
41 } else {
42   message_line.style.display = 'none';
43 }
44 }
```

#8 HIGHSAST_XSSvulnerabilities/api/source/low.php:81

Status: REQUIRES REVIEW (no AI coverage)

SOC2: CC6.6 · CC7.1 CIS: CIS-16.12 · CIS-16.1 ISO: A.8.28 · A.8.26

```
76   const row = tableHead.insertRow(0);
77
78   item = items[0];
79   Object.keys(item).forEach(function(k){
80     let cell = row.insert_th_Cell(-1);
81     cell.innerHTML = k;
82     if (k == 'password') {
83       successDiv = document.getElementById ('message');
84       successDiv.style.display = 'block';
85     }
86   });
87
88   const tableBody = document.getElementById('tableBody');
89
90   items.forEach( item => {
91     let row = tableBody.insertRow();
```

#9 HIGHSAST_XSSvulnerabilities/api/source/low.php:94

Status: REQUIRES REVIEW (no AI coverage)

SOC2: CC6.6 · CC7.1 CIS: CIS-16.12 · CIS-16.1 ISO: A.8.28 · A.8.26

```
89
90 items.forEach( item => {
91   let row = tableBody.insertRow();
92   for (const [key, value] of Object.entries(item)) {
93     let cell = row.insertCell(-1);
94     cell.innerHTML = value;
95   }
96 });
97 }
98 </script>
99 ";
100
101 $html .= "
102
103 <table id='table' class=''>
104 <thead>
```

#10 CRITICALSAST_COMMAND_INJECTIONvulnerabilities/exec/source/impossible.php:22

Status: AI CONFIRMED

SOC2: CC6.6 · CC7.1 CIS: CIS-16.1 · CIS-16.12 · CIS-7.5 ISO: A.8.28 · A.8.26 · A.8.29

AI Analysis: The code is vulnerable to command injection because the \$target variable is directly concatenated into the shell_exec command without any sanitization or validation. This allows an attacker to inject malicious commands, potentially leading to arbitrary code execution.

```
17 $target = $socket[0] . '.' . $socket[1] . '.' . $socket[2] . '.' . $socket[3];
18
19 // Determine OS and execute the ping command.
20 if( strstr( php_uname( 's' ), 'Windows NT' ) ) {
21   // Windows
22   $cmd = shell_exec( 'ping ' . $target );
23 }
24 else {
25   // *nix
26   $cmd = shell_exec( 'ping -c 4 ' . $target );
27 }
28
29 // Feedback for the end user
30 $html .= "<pre>{$cmd}</pre>";
31 }
32 else {
```

#11 CRITICAL

SAST_COMMAND_INJECTION

vulnerabilities/exec/source/impossible.php:26

Status: AI CONFIRMED

SOC2: CC6.6 · CC7.1 CIS: CIS-16.1 · CIS-16.12 · CIS-7.5 ISO: A.8.28 · A.8.26 · A.8.29

AI Analysis: The code is vulnerable to command injection because the \$target variable is directly concatenated into the shell_exec command without any sanitization or validation. This allows an attacker to inject malicious commands, potentially leading to arbitrary code execution.

```
21 // Windows
22 $cmd = shell_exec( 'ping ' . $target );
23 }
24 else {
25 // *nix
26 $cmd = shell_exec( 'ping -c 4 ' . $target );
27 }
28
29 // Feedback for the end user
30 $html .= "<pre>{$cmd}</pre>";
31 }
32 else {
33 // Ops. Let the user name theres a mistake
34 $html .= "<pre>ERROR: You have entered an invalid IP.</pre>";
35 }
36 }
```

#12 CRITICAL

SAST_COMMAND_INJECTION

vulnerabilities/exec/source/high.php:26

Status: AI CONFIRMED

SOC2: CC6.6 · CC7.1 CIS: CIS-16.1 · CIS-16.12 · CIS-7.5 ISO: A.8.28 · A.8.26 · A.8.29

AI Analysis: The code is vulnerable to command injection because it directly executes the 'ping' command with user-controlled input (\$target) without proper sanitization or validation. This allows an attacker to inject malicious commands, potentially leading to arbitrary code execution. The chain context (CHAIN_0003) indicates that this finding is part of a vul...

Chain ID: CHAIN_0003 | Chain Risk: CRITICAL | Rule: secret_to_command_injection

```
21 $target = str_replace( array_keys( $substitutions ), $substitutions, $target );
22
23 // Determine OS and execute the ping command.
24 if( striistr( php_uname( 's' ), 'Windows NT' ) ) {
25 // Windows
26 $cmd = shell_exec( 'ping ' . $target );
27 }
28 else {
29 // *nix
30 $cmd = shell_exec( 'ping -c 4 ' . $target );
31 }
32
33 // Feedback for the end user
34 $html .= "<pre>{$cmd}</pre>";
35 }
36 }
```

#13 CRITICAL

SAST_COMMAND_INJECTION

vulnerabilities/exec/source/high.php:30

Status: AI CONFIRMED

SOC2: CC6.6 · CC7.1 CIS: CIS-16.1 · CIS-16.12 · CIS-7.5 ISO: A.8.28 · A.8.26 · A.8.29

AI Analysis: This finding is a duplicate of the previous one, with the same vulnerability and chain context. The code is still vulnerable to command injection, and the chain context indicates that it is part of a vulnerability chain. The bridge logic is that the output of the first finding (the hardcoded secret) becomes the controlled input for the second findi...

```
25 // Windows
26 $cmd = shell_exec( 'ping ' . $target );
27 }
28 else {
29 // *nix
30 $cmd = shell_exec( 'ping -c 4 ' . $target );
31 }
32
33 // Feedback for the end user
34 $html .= "<pre>{$cmd}</pre>";
35 }
36
37 ?>
```

#14 CRITICAL

SAST_COMMAND_INJECTION

vulnerabilities/exec/source/medium.php:19

Status: AI CONFIRMED

SOC2: CC6.6 · CC7.1 CIS: CIS-16.1 · CIS-16.12 · CIS-7.5 ISO: A.8.28 · A.8.26 · A.8.29

AI Analysis: The code is vulnerable to command injection because it directly executes the 'ping' command with user-controlled input (\$target) without proper sanitization or validation. An attacker could inject malicious commands by manipulating the \$target variable.

```
14 $target = str_replace( array_keys( $substitutions ), $substitutions, $target );
15
16 // Determine OS and execute the ping command.
17 if( striestr( php_uname( 's' ), 'Windows NT' ) ) {
18 // Windows
19 $cmd = shell_exec( 'ping ' . $target );
20 }
21 else {
22 // *nix
23 $cmd = shell_exec( 'ping -c 4 ' . $target );
24 }
25
26 // Feedback for the end user
27 $html .= "<pre>{$cmd}</pre>";
28 }
29
```


#15 CRITICAL

SAST_COMMAND_INJECTION

vulnerabilities/exec/source/medium.php:23

Status: AI CONFIRMED

SOC2: CC6.6 · CC7.1 CIS: CIS-16.1 · CIS-16.12 · CIS-7.5 ISO: A.8.28 · A.8.26 · A.8.29

AI Analysis: The code is vulnerable to command injection because it directly executes the 'ping' command with user-controlled input (\$target) without proper sanitization or validation. An attacker could inject malicious commands by manipulating the \$target variable.

```
18 // Windows
19 $cmd = shell_exec( 'ping ' . $target );
20 }
21 else {
22 // *nix
23 $cmd = shell_exec( 'ping -c 4 ' . $target );
24 }
25
26 // Feedback for the end user
27 $html .= "<pre>{$cmd}</pre>";
28 }
29
30 ?>
```

#16 CRITICAL

SAST_COMMAND_INJECTION

vulnerabilities/exec/source/low.php:10

Status: AI CONFIRMED

SOC2: CC6.6 · CC7.1 CIS: CIS-16.1 · CIS-16.12 · CIS-7.5 ISO: A.8.28 · A.8.26 · A.8.29

AI Analysis: The code is vulnerable to command injection because it directly executes the 'ping' command with user-controlled input (\$target) using shell_exec(). This allows an attacker to inject malicious commands, potentially leading to arbitrary code execution. The lack of input validation or sanitization makes this a critical vulnerability.

```
5 $target = $_REQUEST[ 'ip' ];
6
7 // Determine OS and execute the ping command.
8 if( striistr( php_uname( 's' ), 'Windows NT' ) ) {
9 // Windows
10 $cmd = shell_exec( 'ping ' . $target );
11 }
12 else {
13 // *nix
14 $cmd = shell_exec( 'ping -c 4 ' . $target );
15 }
16
17 // Feedback for the end user
18 $html .= "<pre>{$cmd}</pre>";
19 }
20
```

#17 CRITICAL

SAST_COMMAND_INJECTION

vulnerabilities/exec/source/low.php:14

Status: AI CONFIRMED

SOC2: CC6.6 · CC7.1 CIS: CIS-16.1 · CIS-16.12 · CIS-7.5 ISO: A.8.28 · A.8.26 · A.8.29

AI Analysis: This finding is similar to the previous one, with the same vulnerability and attack vector. The code is still vulnerable to command injection due to the direct execution of the 'ping' command with user-controlled input (\$target) using shell_exec().

```
9 // Windows
10 $cmd = shell_exec( 'ping ' . $target );
11 }
12 else {
13 // *nix
14 $cmd = shell_exec( 'ping -c 4 ' . $target );
15 }
16
17 // Feedback for the end user
18 $html .= "<pre>{$cmd}</pre>";
19 }
20
21 ?>
```

#18 HIGH

SAST_XSS

vulnerabilities/authbypass/authbypass.js:43

Status: REQUIRES REVIEW (no AI coverage)

SOC2: CC6.6 · CC7.1 CIS: CIS-16.12 · CIS-16.1 ISO: A.8.28 · A.8.26

```
38 users.forEach(updateTable);
39
40 function updateTable (user) {
41 var row = table_body.insertRow(0);
42 var cell0 = row.insertCell(-1);
43 cell0.innerHTML = user['user_id'] + '<input type="hidden" id="user_id_' + user['user_id'] + '" name="user_id"
value="' + user['user_id'] + '" />';
44 var cell1 = row.insertCell(1);
45 cell1.innerHTML = '<input type="text" id="first_name_' + user['user_id'] + '" name="first_name" value="' +
user['first_name'] + '" />';
46 var cell2 = row.insertCell(2);
47 cell2.innerHTML = '<input type="text" id="surname_' + user['user_id'] + '" name="surname" value="' +
user['surname'] + '" />';
48 var cell3 = row.insertCell(3);
49 cell3.innerHTML = '<input type="button" value="Update" onclick="submit_change(' + user['user_id'] + ')" />';
50 }
51 };
52 xhr.send();
53 }
```

#19HIGHSAST_XSSvulnerabilities/authbypass/authbypass.js:45

Status: REQUIRES REVIEW (no AI coverage)

SOC2: CC6.6 · CC7.1 CIS: CIS-16.12 · CIS-16.1 ISO: A.8.28 · A.8.26

```
40 function updateTable (user) {
41   var row = table_body.insertRow(0);
42   var cell0 = row.insertCell(-1);
43   cell0.innerHTML = user['user_id'] + '<input type="hidden" id="user_id_' + user['user_id'] + '" name="user_id" value="" + user['user_id'] + '" />';
44   var cell1 = row.insertCell(1);
45   cell1.innerHTML = '<input type="text" id="first_name_' + user['user_id'] + '" name="first_name" value="" + user['first_name'] + '" />';
46   var cell2 = row.insertCell(2);
47   cell2.innerHTML = '<input type="text" id="surname_' + user['user_id'] + '" name="surname" value="" + user['surname'] + '" />';
48   var cell3 = row.insertCell(3);
49   cell3.innerHTML = '<input type="button" value="Update" onclick="submit_change(' + user['user_id'] + ')" />';
50 }
51 };
52 xhr.send();
53 }
```

#20HIGHSAST_XSSvulnerabilities/authbypass/authbypass.js:47

Status: REQUIRES REVIEW (no AI coverage)

SOC2: CC6.6 · CC7.1 CIS: CIS-16.12 · CIS-16.1 ISO: A.8.28 · A.8.26

```
42 var cell0 = row.insertCell(-1);
43 cell0.innerHTML = user['user_id'] + '<input type="hidden" id="user_id_' + user['user_id'] + '" name="user_id" value="" + user['user_id'] + '" />';
44 var cell1 = row.insertCell(1);
45 cell1.innerHTML = '<input type="text" id="first_name_' + user['user_id'] + '" name="first_name" value="" + user['first_name'] + '" />';
46 var cell2 = row.insertCell(2);
47 cell2.innerHTML = '<input type="text" id="surname_' + user['user_id'] + '" name="surname" value="" + user['surname'] + '" />';
48 var cell3 = row.insertCell(3);
49 cell3.innerHTML = '<input type="button" value="Update" onclick="submit_change(' + user['user_id'] + ')" />';
50 }
51 };
52 xhr.send();
53 }
```

#21HIGHSAST_XSSvulnerabilities/authbypass/authbypass.js:49

Status: REQUIRES REVIEW (no AI coverage)

SOC2: CC6.6 · CC7.1 CIS: CIS-16.12 · CIS-16.1 ISO: A.8.28 · A.8.26

```
44  var cell1 = row.insertCell(1);
45  cell1.innerHTML = '<input type="text" id="first_name_' + user['user_id'] + '" name="first_name" value="' +
    user['first_name'] + '" />';
46  var cell2 = row.insertCell(2);
47  cell2.innerHTML = '<input type="text" id="surname_' + user['user_id'] + '" name="surname" value="' +
    user['surname'] + '" />';
48  var cell3 = row.insertCell(3);
49  cell3.innerHTML = '<input type="button" value="Update" onclick="submit_change(' + user['user_id'] + ')" />';
50  }
51  };
52  xhr.send();
53  }
```

#22HIGHSAST_XSSvulnerabilities/xss_d/index.php:53

Status: REQUIRES REVIEW (no AI coverage)

SOC2: CC6.6 · CC7.1 CIS: CIS-16.12 · CIS-16.1 ISO: A.8.28 · A.8.26

```
48  <form name="XSS" method="GET">
49  <select name="default">
50  <script>
51  if (document.location.href.indexOf("default=") >= 0) {
52  var lang = document.location.href.substring(document.location.href.indexOf("default=")+8);
53  document.write("<option value='" + lang + "'" + $decodeURI(lang) + "</option>");
54  document.write("<option value=' ' disabled='disabled'>----</option>");
55  }
56
57  document.write("<option value='English'>English</option>");
58  document.write("<option value='French'>French</option>");
59  document.write("<option value='Spanish'>Spanish</option>");
60  document.write("<option value='German'>German</option>");
61  </script>
62  </select>
63  <input type="submit" value="Select" />
```

#23HIGH

SAST_XSS

vulnerabilities/xss_d/index.php:54

Status: REQUIRES REVIEW (no AI coverage)

SOC2: CC6.6 · CC7.1 CIS: CIS-16.12 · CIS-16.1 ISO: A.8.28 · A.8.26

```
49  <select name="default">
50  <script>
51  if (document.location.href.indexOf("default=") >= 0) {
52  var lang = document.location.href.substring(document.location.href.indexOf("default=")+8);
53  document.write("<option value='" + lang + "'" + $decodeURI(lang) + "</option>");
54  document.write("<option value=' ' disabled='disabled'>----</option>");
55  }
56
57  document.write("<option value='English'>English</option>");
58  document.write("<option value='French'>French</option>");
59  document.write("<option value='Spanish'>Spanish</option>");
60  document.write("<option value='German'>German</option>");
61  </script>
62  </select>
63  <input type="submit" value="Select" />
64  </form>
```

#24HIGH

SAST_XSS

vulnerabilities/xss_d/index.php:57

Status: REQUIRES REVIEW (no AI coverage)

SOC2: CC6.6 · CC7.1 CIS: CIS-16.12 · CIS-16.1 ISO: A.8.28 · A.8.26

```
52  var lang = document.location.href.substring(document.location.href.indexOf("default=")+8);
53  document.write("<option value='" + lang + "'" + $decodeURI(lang) + "</option>");
54  document.write("<option value=' ' disabled='disabled'>----</option>");
55  }
56
57  document.write("<option value='English'>English</option>");
58  document.write("<option value='French'>French</option>");
59  document.write("<option value='Spanish'>Spanish</option>");
60  document.write("<option value='German'>German</option>");
61  </script>
62  </select>
63  <input type="submit" value="Select" />
64  </form>
65  </div>
66  EOF;
67
```

#25HIGHSAST_XSSvulnerabilities/xss_d/index.php:58

Status: REQUIRES REVIEW (no AI coverage)

SOC2: CC6.6 · CC7.1 CIS: CIS-16.12 · CIS-16.1 ISO: A.8.28 · A.8.26

```
53 document.write("<option value='" + lang + "'" + $decodeURI(lang) + "</option>");
54 document.write("<option value=' ' disabled='disabled'>----</option>");
55 }
56
57 document.write("<option value='English'>English</option>");
58 document.write("<option value='French'>French</option>");
59 document.write("<option value='Spanish'>Spanish</option>");
60 document.write("<option value='German'>German</option>");
61 </script>
62 </select>
63 <input type="submit" value="Select" />
64 </form>
65 </div>
66 EOF;
67
68 $page[ 'body' ] .= "
```

#26HIGHSAST_XSSvulnerabilities/xss_d/index.php:59

Status: REQUIRES REVIEW (no AI coverage)

SOC2: CC6.6 · CC7.1 CIS: CIS-16.12 · CIS-16.1 ISO: A.8.28 · A.8.26

```
54 document.write("<option value=' ' disabled='disabled'>----</option>");
55 }
56
57 document.write("<option value='English'>English</option>");
58 document.write("<option value='French'>French</option>");
59 document.write("<option value='Spanish'>Spanish</option>");
60 document.write("<option value='German'>German</option>");
61 </script>
62 </select>
63 <input type="submit" value="Select" />
64 </form>
65 </div>
66 EOF;
67
68 $page[ 'body' ] .= "
69 <h2>More Information</h2>
```

#27 HIGH	SAST_XSS	vulnerabilities/xss_d/index.php:60
Status: REQUIRES REVIEW (no AI coverage)		
SOC2: CC6.6 · CC7.1 CIS: CIS-16.12 · CIS-16.1 ISO: A.8.28 · A.8.26		
<pre>55 } 56 57 document.write("<option value='English'>English</option>"); 58 document.write("<option value='French'>French</option>"); 59 document.write("<option value='Spanish'>Spanish</option>"); 60 document.write("<option value='German'>German</option>"); 61 </script> 62 </select> 63 <input type="submit" value="Select" /> 64 </form> 65 </div> 66 EOF; 67 68 \$page['body'] .= " 69 <h2>More Information</h2> 70 </pre>		
#28 CRITICAL	SAST_COMMAND_INJECTION	vulnerabilities/javascript/source/high.js:1
Status: AI DISMISSED — MANUAL REVIEW ADVISED		
SOC2: CC6.6 · CC7.1 CIS: CIS-16.1 · CIS-16.12 · CIS-7.5 ISO: A.8.28 · A.8.26 · A.8.29		
AI Analysis: The provided code is heavily obfuscated and does not clearly demonstrate a command injection vulnerability. The code appears to be implementing a SHA-256 hash function, and there is no clear indication of user input being used to construct a command. Without further context or a clear example of how user input is being used, it is difficult to dete...		
<pre>1 var a=['fromCharCode','toString','replace','BeJ','\x5cw+', 'Lyg', 'SuR', '(w){\x273M\x203L\x27;q\x201l=\x273K\x203I \x203J\x20T\x27;q\x201R=1c\x202I==\x271n\x27;q</pre>		

#29 CRITICAL

SAST_COMMAND_INJECTION

vulnerabilities/javascript/source/high_unobfuscated.js:83

Status: AI DISMISSED — MANUAL REVIEW ADVISED

SOC2: CC6.6 · CC7.1 CIS: CIS-16.1 · CIS-16.12 · CIS-7.5 ISO: A.8.28 · A.8.26 · A.8.29

AI Analysis: The provided code does not demonstrate a command injection vulnerability. The code is using the `crypto` module to create a hash, and the `eval` function is used to require the `crypto` and `buffer` modules. However, there is no clear indication of user input being used to construct a command. The code appears to be a Node.js implementation of a SH...

```
78  }
79  return method;
80  };
81
82  var nodeWrap = function (method, is224) {
83    var crypto = eval("require('crypto')");
84    var Buffer = eval("require('buffer').Buffer");
85    var algorithm = is224 ? 'sha224' : 'sha256';
86    var nodeMethod = function (message) {
87      if (typeof message === 'string') {
88        return crypto.createHash(algorithm).update(message, 'utf8').digest('hex');
89      } else {
90        if (message === null || message === undefined) {
91          throw new Error(ERROR);
92        } else if (message.constructor === ArrayBuffer) {
93          message = new Uint8Array(message);
```

#30 CRITICAL

SAST_COMMAND_INJECTION

vulnerabilities/javascript/source/high_unobfuscated.js:84

Status: AI DISMISSED — MANUAL REVIEW ADVISED

SOC2: CC6.6 · CC7.1 CIS: CIS-16.1 · CIS-16.12 · CIS-7.5 ISO: A.8.28 · A.8.26 · A.8.29

AI Analysis: The provided code does not contain any user-controlled input that could lead to command injection. The 'message' variable is checked for its type and handled accordingly, but there is no indication that it comes from an untrusted source. Additionally, the 'crypto.createHash' function is used with a fixed algorithm, which does not allow for command ...

```
79  return method;
80  };
81
82  var nodeWrap = function (method, is224) {
83    var crypto = eval("require('crypto')");
84    var Buffer = eval("require('buffer').Buffer");
85    var algorithm = is224 ? 'sha224' : 'sha256';
86    var nodeMethod = function (message) {
87      if (typeof message === 'string') {
88        return crypto.createHash(algorithm).update(message, 'utf8').digest('hex');
89      } else {
90        if (message === null || message === undefined) {
91          throw new Error(ERROR);
92        } else if (message.constructor === ArrayBuffer) {
93          message = new Uint8Array(message);
94        }
```


#31 CRITICAL

SAST_COMMAND_INJECTION

vulnerabilities/cryptography/source/oracle_attack.php:57

Status: AI CONFIRMED

SOC2: CC6.6 · CC7.1 CIS: CIS-16.1 · CIS-16.12 · CIS-7.5 ISO: A.8.28 · A.8.26 · A.8.29

AI Analysis: The provided code is part of a vulnerability chain (CHAIN_0003) with a critical risk. The 'curl_exec' function is used to execute a curl request, and the '\$url' variable is not sanitized. If an attacker can control the '\$url' variable, they could inject malicious commands. The bridge logic is that the output of the first finding (e.g., a hardcoded ...

Chain ID: CHAIN_0003 | Chain Risk: CRITICAL | Rule: secret_to_command_injection

```
52  curl_setopt($ch, CURLOPT_POSTFIELDS, json_encode ($data));
53
54  curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
55  curl_setopt($ch, CURLOPT_HEADER, true);
56
57  $response = curl_exec($ch);
58
59  $header_size = curl_getinfo($ch, CURLINFO_HEADER_SIZE);
60  $header = substr($response, 0, $header_size);
61  $body = substr($response, $header_size);
62
63  // May return false or something that evaluates to false
64  // so can't do strict type check
65  if ($response == false) {
66      print "Could not access remote server, is the URL correct?
67      ${url}
```

#32 HIGH

SAST_XSS

vulnerabilities/csp/source/high.js:9

Status: REQUIRES REVIEW (no AI coverage)

SOC2: CC6.6 · CC7.1 CIS: CIS-16.12 · CIS-16.1 ISO: A.8.28 · A.8.26

4 document.body.appendChild(s);
5 }
6
7 function solveSum(obj) {
8 if ("answer" in obj) {
9 document.getElementById("answer").innerHTML = obj['answer'];
10 }
11 }
12
13 var solve_button = document.getElementById ("solve");
14
15 if (solve_button) {
16 solve_button.addEventListener("click", function() {
17 clickButton();
18 });
19 }

#33HIGH

SAST_XSS

vulnerabilities/csp/source/impossible.js:9

Status: REQUIRES REVIEW (no AI coverage)

SOC2: CC6.6 · CC7.1 CIS: CIS-16.12 · CIS-16.1 ISO: A.8.28 · A.8.26

```
4  document.body.appendChild(s);
5  }
6
7  function solveSum(obj) {
8    if ("answer" in obj) {
9      document.getElementById("answer").innerHTML = obj['answer'];
10   }
11   }
12
13   var solve_button = document.getElementById ("solve");
14
15   if (solve_button) {
16     solve_button.addEventListener("click", function() {
17       clickButton();
18     });
19   }
```